Module 09 — Spring Security Lite

Phase: Backend (who may call the API)
Previous module: REST, DTOs, and Postman
Next module: Lab: CRUD + Auth Consolidation
Lab kit: 15_Lab_Assets/19_spring_auth_tasks/

Learning Outcomes

By the end of this module, learners will be able to:

1. Separate **authentication** (who you are) from **authorization** (what you may do).
2. Store passwords with **BCrypt** — never the raw string — using `PasswordEncoder`.
3. Add `spring-boot-starter-security` and a Boot 3 `SecurityFilterChain` (no `WebSecurityConfigurerAdapter`).
4. Register and login a CampusCart user (`POST /auth/register`, `POST /auth/login`) and return a **JWT**.
5. Send `Authorization: Bearer <token>` and **protect** `POST /products` while **GET** stays public.
6. Keep JWT secret and DB password in **env** — not in git.

Teaching note: Modules 03–04 already persist a **DTO catalog** in MySQL. **Do not rewrite JPA / mapping / `@Valid` / Postman v1.** Today the catalog gets a **door**. Anyone may **read** snacks. Only a logged-in user may **add** one. No OAuth2, no refresh tokens, no admin-role matrix. No Lombok. No kit 20. Protected lab days that glue CRUD + auth are **Later —** the next file. React login UI is **Later —** React weeks.

Teach order today: AuthN vs AuthZ → BCrypt → `SecurityFilterChain` → `AppUser` + register/login → JWT Bearer → protect POST.

Prerequisites

- **CampusCart DTOs + Postman** from REST, DTOs, and Postman — `ProductRequest` / `ProductResponse`, `@Valid`, lite 400, MySQL catalog. JDK 17 + Maven 3.9+ + Spring Boot 3.x (Tested Toolchain).
- **Postman** from the previous module. You will add **Authorization → Bearer Token**.

Definition / Purpose

Definition: Authentication (AuthN) answers **who is calling**. CampusCart login checks email + password. **Authorization (AuthZ)** answers **what that person may do**. Here: guests **GET** the menu; only a logged-in user **POST**s a snack.

Definition: BCrypt is a slow **hash** for passwords. You store the hash. You never store `IdliDosa8`. `PasswordEncoder.matches(raw, hash)` checks a login.

Definition: A **JWT** (JSON Web Token) is a **signed** string the server gives after login. The client sends it on later requests as `Authorization: Bearer …`. The server **verifies the signature** — it does not look up a server session (stateless).

Definition: `SecurityFilterChain` is the Boot 3 bean that replaces the old `WebSecurityConfigurerAdapter`. You list which URLs are **public** and which need a user.

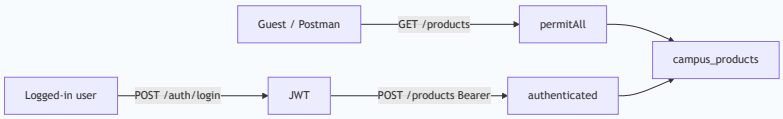
Purpose: Ravi POSTed Pani puri from a random laptop. The canteen menu filled with junk. Priya needs a **login**. Interviews will ask AuthN vs AuthZ and why hashes beat plain text. **Later —** Google login / refresh tokens / admin roles.

Bank | Order | Laptop | Employee | UPI

Where used?

- CampusCart — public menu, logged-in add-snack
- TechMart — public catalog, signed-in cart (**Later —** cart)
- SecureBank — you must prove who you are before a transfer (**no money movement here**)
- AcmeHR — staff login before leave apply
- MedCare — public doctor list; booking needs a login (**Later —**)

Visual — public GET, protected POST:



Key takeaways:

- **AuthN** = identity. **AuthZ** = permission
- **BCrypt** in the table; raw password only in the POST body (in memory)
- Boot 3: `SecurityFilterChain`, not the old adapter class
- **JWT** in the **header**, secret in **env**
- GET list stays public on the core path; POST needs a token

Motivation — Feel the pain of an open POST

Anyone who guessed `POST /products` added “Free pizza”. Workbench showed rows Priya never typed. GET is a menu. POST is a **write**. Writes need a known user.

```
public class OpenPostPain {
    public static void main(String[] args) {
        System.out.println("GET /products is a public menu");
        System.out.println("POST /products changed MySQL with no login");
        System.out.println("Need AuthN then AuthZ on create");
    }
}
```

Expected Output:

```
GET /products is a public menu
POST /products changed MySQL with no login
Need AuthN then AuthZ on create
```

Remember: This module does not replace DTOs. It **guards** the create mapping you already wrote.

Interview: “Is login authorization?” — Login is **authentication**. Deciding that only authenticated callers may POST is **authorization**.

Important Concepts

1. AuthN vs AuthZ

Definition: AuthN proves identity (email + password → user). **AuthZ** checks permission (this user may POST `/products`).

Purpose: Mixing the words in interviews loses marks. A 401 means “we do not know you.” A 403 means “we know you; you still may not.” This module’s core path uses **401** for missing/bad tokens. Role-based 403 is **Later —**.

Status	Meaning on this course
401	Not authenticated (no/invalid JWT, bad login)
403	Authenticated but not allowed (**Later —** roles)
400	Bad JSON / validation (module 04)
404	Unknown product id (modules 02–04)

```
public class AuthnAuthzLine {
    public static void main(String[] args) {
        System.out.println("AuthN: Priya logged in");
        System.out.println("AuthZ: Priya may POST /products");
        System.out.println("401 = no valid token");
    }
}
```

Expected Output:

```
AuthN: Priya logged in
AuthZ: Priya may POST /products
401 = no valid token
```

Note:
- Wrong: calling every security error “403”.
- Wrong: hiding the product GET behind login on the core path — the menu is public.
- Prefer: 401 for anonymous POST; keep GET public.

Common Error: Returning 200 with `"error":"bad password"` — that is still a failed AuthN. Use **401**.

Remember: Public **read**, protected **write** is a common shop API.

Interview: AuthN vs AuthZ in one sentence each.

Try these (2 programs):

Program 1 — Two questions

```
public class TwoSecurityQuestions {
    public static void main(String[] args) {
        System.out.println("Who are you? -> AuthN");
        System.out.println("What may you do? -> AuthZ");
    }
}
```

Expected Output:

```
Who are you? -> AuthN
What may you do? -> AuthZ
```

Program 2 — Catalog rules

```
public class CatalogRulesPrint {
    public static void main(String[] args) {
        System.out.println("GET /products -> public");
        System.out.println("POST /products -> JWT required");
        System.out.println("POST /auth/login -> public");
    }
}
```

Expected Output:

```
GET /products -> public
POST /products -> JWT required
POST /auth/login -> public
```

---

2. BCrypt — never store the raw password

Definition: Hashing turns a password into a **one-way** string. **BCrypt** adds a **salt** so two users with `IdliDosa8` do not share the same hash. Spring's **`BCryptPasswordEncoder`** implements **`PasswordEncoder`**.

Purpose: If MySQL leaks, attackers must still crack hashes. Plain `password` columns fail the interview and the audit.

```
package in.ksr.campusauth;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
public class PasswordConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

Usage (inside register / login — Maven tree):

```
String hash = encoder.encode(rawPassword);
boolean ok = encoder.matches(rawPassword, hash);
```

Hashes usually start with `$2a$` or `$2b$`.

Note:
- Wrong: `NoOpPasswordEncoder` / saving `incoming.getPassword()` as the column.
- Wrong: logging the raw password.
- Prefer: encode on register; `matches` on login; same failure message if email is unknown or password is wrong.

Common Error: `encode` on every login — that produces a **new** hash and never matches the stored one. Login uses `matches`.

Remember: HTTPS carries the raw password **once** on POST. The table stores only the hash.

Interview: Why not SHA-256 once? Fast hashes are easier to brute-force. BCrypt is designed to be **slow**.

Try these (2 programs):

Program 1 — What the column holds

```
public class HashColumnPrint {
    public static void main(String[] args) {
        System.out.println("POST body: IdliDosa8");
        System.out.println("MySQL column: $2a$10$...");
        System.out.println("never store IdliDosa8");
    }
}
```

Expected Output:

```
POST body: IdliDosa8
MySQL column: $2a$10$...
never store IdliDosa8
```

Program 2 — encode vs matches

```
public class EncodeVsMatches {
    public static void main(String[] args) {
        System.out.println("register -> encoder.encode");
        System.out.println("login    -> encoder.matches");
    }
}
```

Expected Output:

```
register -> encoder.encode
login    -> encoder.matches
```

3. `SecurityFilterChain` — Boot 3 DSL

Definition: `SecurityFilterChain` is a `@Bean` that configures **CSRF**, **session**, and **URL rules**. Boot 3 removed `WebSecurityConfigurerAdapter`. Tutorials that `extends` that class **will not compile** on this parent.

Purpose: Without this bean, starter-security locks **every** URL and prints a **generated password**. You would not be able to register.

Copy-paste valid (kit package):

```
package in.ksr.campusauth;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
public class SecurityConfigSketch {

    private final JwtAuthFilter jwtAuthFilter;

    public SecurityConfigSketch(JwtAuthFilter jwtAuthFilter) {
        this.jwtAuthFilter = jwtAuthFilter;
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.csrf(csrf -> csrf.disable())
                .sessionManagement(sm ->
                        sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/auth/**").permitAll()
                        .requestMatchers(HttpMethod.GET, "/products", "/products/**")
                        .permitAll()
                        .anyRequest().authenticated())
                .addFilterBefore(
                        jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}
```

Why CSRF off? CSRF targets **cookie** sessions in browsers. This API uses a **Bearer token** the JS client must attach. Cookie-session CSRF is **Later —** if you switch to cookies.

Why STATELESS? JWT is the memory of login. Do not also create an HTTP session.

> **Note:**
> - Wrong: `WebSecurityConfigurerAdapter`.
> - Wrong: `anyRequest().permitAll()` — that is the kit **broken** tree.
> - Prefer: list public matchers **before** `anyRequest().authenticated()`.

> **Common Error:** Forgetting `/auth/**` as `permitAll` — register returns 401 before you can create a user.

> **Remember:** Order of matchers matters. First match wins.

> **Interview:** Why disable CSRF on a JWT API? Because the browser is not auto-sending a session cookie on cross-site POST for this design.

Try these (2 programs):

Program 1 — Public matchers

```
public class PublicMatchersPrint {
    public static void main(String[] args) {
        System.out.println("/auth/** permitAll");
        System.out.println("GET /products permitAll");
        System.out.println("POST /products authenticated");
    }
}
```

Expected Output:

```
/auth/** permitAll
GET /products permitAll
POST /products authenticated
```

Program 2 — Dead class name

```
public class AdapterIsDead {
    public static void main(String[] args) {
        System.out.println("Boot 3: SecurityFilterChain @Bean");
        System.out.println("Not: WebSecurityConfigurerAdapter");
    }
}
```

Expected Output:

```
Boot 3: SecurityFilterChain @Bean
Not: WebSecurityConfigurerAdapter
```

---

4. `AppUser` — register and login

Definition: `AppUser` is a JPA entity for `campus_users`: unique **email**, `passwordHash`. It is **not** the Product table. JSON in is `RegisterRequest` / `LoginRequest`. JSON out is `AuthResponse` (`token`, `email`) — **never** the hash.

Purpose: Product rows are snacks. User rows are people. Do not hang a password on `Product`.

Entity sketch:

```
@Entity
@Table(name = "campus_users")
public class AppUser {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(name = "password_hash", nullable = false)
```

```
    private String passwordHash;
    // getters / setters
}
```

Register (happy path): lowercase email → if exists **409** → `encode` → `save` → JWT → **201**.
Login: find by email → `matches` → JWT **200** or **401** `"Bad email or password"` (same text either way).

Note:
- Wrong: returning the entity (hash leaks).
- Wrong: different messages for “unknown email” vs “bad password” (account enumeration).
- Prefer: `@Email` + `@Size(min = 8)` on register password.

Common Error: Unique email only in Java. Add `unique = true` on the column so MySQL also refuses duplicates.

Remember: `campus_users` is a new table. Do not drop MYSQL homework `users`.

Interview: Why not `User` as the class name? It collides with `java.lang` / Security’s `User` builder. `AppUser` stays clear.

Try these (2 programs):

Program 1 — Two tables

```
public class TwoTablesPrint {
    public static void main(String[] args) {
        System.out.println("campus_products -> snacks");
        System.out.println("campus_users -> email + hash");
    }
}
```

Expected Output:

```
campus_products -> snacks
campus_users -> email + hash
```

Program 2 — Auth JSON out

```
public class AuthJsonOut {
    public static void main(String[] args) {
        System.out.println("token");
        System.out.println("email");
        System.out.println("no passwordHash");
    }
}
```

Expected Output:

```
token
email
no passwordHash
```

5. JWT — Bearer header

Definition: A **JWT** has three Base64 parts: **header.payload.signature**. This course uses **HS256** and a **server secret**. **`jjwt` 0.12** (`Jwts.builder()`, `parser().verifyWith(key)`).

Purpose: After login, Postman should not resend the password on every POST. It sends the token. React weeks will do the same in `fetch` headers (**Later —**).

Header:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
```

Issue token (kit `JwtService`):

```
return Jwts.builder()
        .subject(email)
        .issuedAt(new Date())
        .expiration(exp)
        .signWith(key)
        .compact();
```

Secret: **`JWT_SECRET`** env, **≥ 32 characters** for HS256. yml: `app.jwt.secret: ${JWT_SECRET}`.

Note:
- Wrong: putting the secret in `application.yml` as a literal (kit **broken**).
- Wrong: sending the token as a query parameter `?token=`.
- Prefer: Authorization header only.

Common Error: Secret shorter than 32 bytes → jjwt throws at startup. Use the kit `.env.example` dummy and **change it**.

Remember: Anyone who **steals** a valid token is you until it expires. Short expiry (60 minutes) is enough for class. Refresh tokens are **Later —**.

Interview: JWT vs session cookie? JWT is sent by the client each time (stateless). A session id is looked up in server memory or Redis.

Try these (2 programs):

Program 1 — Header shape

```
public class BearerHeaderPrint {
    public static void main(String[] args) {
        System.out.println("Authorization: Bearer <jwt>");
        System.out.println("not ?token=");
    }
}
```

Expected Output:

```
Authorization: Bearer <jwt>
not ?token=
```

Program 2 — Secret home

```
public class JwtSecretHome {
    public static void main(String[] args) {
        System.out.println("yml: ${JWT_SECRET}");
        System.out.println("git: .env.example YES");
        System.out.println("git: real secret NO");
    }
}
```

Expected Output:

```
yml: ${JWT_SECRET}
git: .env.example YES
git: real secret NO
```

---

6. Filter — POST needs a user

Definition: `JwtAuthFilter` extends `OncePerRequestFilter`. If the header is `Bearer …`, it **parses** the email, loads `UserDetails`, and sets `SecurityContext`. Then the chain sees an **authenticated** user.

Purpose: `SecurityFilterChain` only knows "this URL needs authentication." The filter **turns a token into a user**. Without it, even a valid JWT looks anonymous → **401**.

```
String header = request.getHeader(HttpHeaders.AUTHORIZATION);
if (header != null && header.startsWith("Bearer ")) {
    String email = jwtService.parseEmail(header.substring(7));
    UserDetails details = users.loadUserByUsername(email);
    UsernamePasswordAuthenticationToken auth =
            new UsernamePasswordAuthenticationToken(
                    details, null, details.getAuthorities());
    SecurityContextHolder.getContext().setAuthentication(auth);
}
filterChain.doFilter(request, response);
```

Register the filter with `addFilterBefore(..., UsernamePasswordAuthenticationFilter.class)`.

Lite 401 body: `{"error":"Unauthorized"}` from an `AuthenticationEntryPoint` — same family as module 04's small JSON, different status.

Note:
- Wrong: skipping `filterChain.doFilter` — the request dies.
- Wrong: treating GET `/products` as authenticated-only on the core path.
- Prefer: invalid token → clear context → chain continues → 401 on protected URLs.

Common Error: Postman **Inherit auth from parent** empty, or typing `Bearer` twice (`Bearer Bearer eyJ…`).

Remember: curl/Postman 401 on POST is **success of the guard**, not a broken GET.

Interview: Where is the user stored between requests? In the **token**, not in Tomcat session (`STATELESS`).

Try these (2 programs):

Program 1 — POST without token

```
public class PostWithoutToken {
    public static void main(String[] args) {
        System.out.println("POST /products no header");
        System.out.println("401 Unauthorized");
    }
}
```

Expected Output:

```
POST /products no header
401 Unauthorized
```

Program 2 — POST with token

```
public class PostWithToken {
    public static void main(String[] args) {
        System.out.println("Authorization Bearer jwt");
        System.out.println("201 ProductResponse");
    }
}
```

Expected Output:

```
Authorization Bearer jwt
201 ProductResponse
```

Step-by-Step Annotated Java Code

Work in lab kit **19_spring_auth_tasks** (`starter/`). JDK 17. Open the folder with **`pom.xml`**. Product DTO/JPA TODOs from kit 18 are **already filled**. Export `.env.example` including **`JWT_SECRET`** (32+ chars).

Step 1: Prove the catalog still serves GET

```
export MYSQL_PASSWORD='your_lab_password'
export JWT_SECRET='change_me_lab_jwt_secret_32chars!'
cd starter
mvn spring-boot:run
```

```
curl http://localhost:8080/products
```

If Security blocks GET before you finish matchers, that is expected — Step 3 opens GET.

Try these (2 programs):

Program 1 — Catalog still MySQL

```
public class CatalogStillMysql {
    public static void main(String[] args) {
        System.out.println("Product entity unchanged");
```

```
        System.out.println("findAll still GET list");
    }
}
```

Expected Output:

```
Product entity unchanged
findAll still GET list
```

Program 2 — New starter

```
public class NewStarterPrint {
    public static void main(String[] args) {
        System.out.println("spring-boot-starter-security");
        System.out.println("jjwt-api / jjwt-impl / jjwt-jackson");
    }
}
```

Expected Output:

```
spring-boot-starter-security
jjwt-api / jjwt-impl / jjwt-jackson
```

Step 2: `AppUser` + BCrypt register

Complete `AppUser` unique email. `PasswordEncoder` bean. `POST /auth/register` encodes, saves, returns token. Check Workbench: `password_hash` starts with `$2`.

Try these (2 programs):

Program 1 — Register status

```
public class RegisterStatusPrint {
    public static void main(String[] args) {
        System.out.println("POST /auth/register -> 201");
        System.out.println("duplicate email -> 409");
    }
}
```

Expected Output:

```
POST /auth/register -> 201
duplicate email -> 409
```

Program 2 — Curl register

```
public class CurlRegisterPrint {
    public static void main(String[] args) {
        System.out.println("curl -X POST http://localhost:8080/auth/register");
        System.out.println("JSON email + password");
```

```
    }
}
```

Expected Output:

```
curl -X POST http://localhost:8080/auth/register
JSON email + password
```

Step 3: `SecurityFilterChain` public vs authenticated

Permit `/auth/**` and GET `/products`. `anyRequest().authenticated()`. CSRF off. STATELESS. GET list should work without a token. POST without token → 401.

Try these (2 programs):

Program 1 — Session policy

```
public class StatelessPrint {
    public static void main(String[] args) {
        System.out.println("SessionCreationPolicy.STATELESS");
        System.out.println("JWT holds login");
    }
}
```

Expected Output:

```
SessionCreationPolicy.STATELESS
JWT holds login
```

Program 2 — CSRF off reason

```
public class CsrfOffPrint {
    public static void main(String[] args) {
        System.out.println("Bearer token API");
        System.out.println("csrf.disable for this design");
    }
}
```

Expected Output:

```
Bearer token API
csrf.disable for this design
```

Step 4: Login + real JWT

`JwtService.createToken` / `parseEmail`. Login `matches` then 200 token. Bad password 401. Secret from env.

Try these (2 programs):

Program 1 — Login statuses

```
public class LoginStatusesPrint {
    public static void main(String[] args) {
        System.out.println("good password -> 200 token");
```

```
        System.out.println("bad password  -> 401");
    }
}
```

Expected Output:

```
good password -> 200 token
bad password  -> 401
```

Program 2 — Three JWT parts

```
public class JwtThreeParts {
    public static void main(String[] args) {
        System.out.println("header.payload.signature");
    }
}
```

Expected Output:

```
header.payload.signature
```

Step 5: Filter + Bearer POST

`JwtAuthFilter` reads Bearer. `addFilterBefore`. POST with token → 201 DTO (same as module 04). POST without → 401 JSON.

```
curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:8080/products \
  -H "Content-Type: application/json" \
  -d '{"name":"Pani puri","priceInr":30}'
```

Expected Output:

```
401
```

Try these (2 programs):

Program 1 — Filter class

```
public class FilterClassPrint {
    public static void main(String[] args) {
        System.out.println("OncePerRequestFilter");
        System.out.println("Bearer substring 7");
    }
}
```

Expected Output:

```
OncePerRequestFilter
Bearer substring 7
```

Program 2 — Entry point JSON

```
public class EntryPointJson {
    public static void main(String[] args) {
        System.out.println("{\"error\":\"Unauthorized\"}");
    }
}
```

Expected Output:

```
{"error":"Unauthorized"}
```

Step 6: Postman + git hygiene

Import `CampusCart_Auth_v1.postman_collection.json`. Register → token variable → POST with Bearer. `git status` must not list `.env`. README: env, run, curl 401 then 201.

Do **not** add OAuth2, refresh tokens, roles, or Lombok to pass.

Try these (2 programs):

Program 1 — Collection file

```
public class AuthCollectionFile {
    public static void main(String[] args) {
        System.out.println("CampusCart_Auth_v1.postman_collection.json");
    }
}
```

Expected Output:

```
CampusCart_Auth_v1.postman_collection.json
```

Program 2 — Git hygiene

```
public class AuthGitHygiene {
    public static void main(String[] args) {
        System.out.println("commit collection json");
        System.out.println("do not commit JWT_SECRET");
        System.out.println("do not commit target/");
    }
}
```

Expected Output:

```
commit collection json
do not commit JWT_SECRET
do not commit target/
```

Practice Exercises

(Each Practice item includes a filled Model answer: and Expected Output: where the demo prints.)

Basic

1. Print AuthN vs AuthZ in two lines.

Model answer:

```
public class AuthnAuthzTwoLines {
    public static void main(String[] args) {
        System.out.println("AuthN: prove identity");
        System.out.println("AuthZ: allow POST or not");
    }
}
```

Expected Output:

```
AuthN: prove identity
AuthZ: allow POST or not
```

2. Print what BCrypt stores.

Model answer:

```
public class BcryptStoresPrint {
    public static void main(String[] args) {
        System.out.println("hash + salt in password_hash");
        System.out.println("not the raw password");
    }
}
```

Expected Output:

```
hash + salt in password_hash
not the raw password
```

3. Print the Bearer header.

Model answer:

```
public class BearerLinePrint {
    public static void main(String[] args) {
        System.out.println("Authorization: Bearer <token>");
    }
}
```

Expected Output:

```
Authorization: Bearer <token>
```

Intermediate

1. Why GET stays public.

Model answer:

```
public class WhyGetPublic {
    public static void main(String[] args) {
        System.out.println("Menu is for every student");
        System.out.println("Create is a write");
        System.out.println("Writes need AuthN");
    }
}
```

Expected Output:

```
Menu is for every student
Create is a write
Writes need AuthN
```

2. Why duplicate email is 409 not 201.

Model answer:

```
public class DuplicateEmail409 {
    public static void main(String[] args) {
        System.out.println("unique email column");
        System.out.println("second register is a conflict");
        System.out.println("409");
    }
}
```

Expected Output:

```
unique email column
second register is a conflict
409
```

3. Why the JWT secret is 32+ characters.

Model answer:

```
public class SecretLengthPrint {
    public static void main(String[] args) {
        System.out.println("HS256 key must be 256 bits");
        System.out.println("32 ASCII chars is the lab minimum");
    }
}
```

Expected Output:

```
HS256 key must be 256 bits
32 ASCII chars is the lab minimum
```

4. Why this is not OAuth2.

Model answer:

```
public class NotOauth2Print {
    public static void main(String[] args) {
        System.out.println("We issue our own HS256 JWT");
        System.out.println("OAuth2 / Google login is Later");
    }
}
```

Expected Output:

```
We issue our own HS256 JWT
OAuth2 / Google login is Later
```

Interview-Oriented

1. AuthN vs AuthZ?

Model answer: Authentication identifies the caller (login / JWT). Authorization decides whether that caller may POST `/products`.

2. Why BCrypt not plain text?

Model answer: A leaked table must not reveal passwords. BCrypt is a slow salted hash. Login uses `matches`, not a second `encode` compared with `==`.

3. Why not `WebSecurityConfigurerAdapter`?

Model answer: Removed in Spring Security 6 / Boot 3. Use a `SecurityFilterChain` `@Bean` and the lambda DSL.

4. 401 vs 404 vs 400?

Model answer: 401 = not logged in / bad token. 404 = product id missing (already taught). 400 = validation on a DTO.

5. Is CORS required to pass?

Model answer: No. Stretch only (`localhost:5173`). React `fetch` is **Later —**.

6. Does JWT replace JPA?

Model answer: No. Same `Product` + DTOs + MySQL. Security decides **who** may call POST.

Mini-Project (Assignment): CampusCart JWT Auth Lite

Lab kit: 15_Lab_Assets/19_spring_auth_tasks/

Modules 03–04 persist a validated catalog. This assignment is the **portfolio auth spike**: **register / login**, **BCrypt**, **JWT**, **GET public**, **POST protected**. Push a public repo. No secrets in git.

Do **not** add OAuth2, refresh tokens, role matrices, or Lombok to pass. CORS is Stretch.

What to build

Piece	Detail
Persistence	Keep kit 18 **Product** DTO GET/POST/404/400 on MySQL

Piece	Detail
`pom.xml`	Add `spring-boot-starter-security` + **jjwt** 0.12 (`jjwt-api` / `impl` / `jackson`)
`AppUser`	`campus_users`, unique email, `passwordHash`
Register	**POST** `/auth/register` → **201** `token` + `email`; duplicate **409**
Login	**POST** `/auth/login` → **200** token or **401**
Chain	CSRF off, **STATELESS**, permit `/auth/**` and GET `/products`
POST catalog	**401** without Bearer; **201** with valid JWT
Secret	`app.jwt.secret: ${JWT_SECRET}` (≥ 32 chars)
Postman	`CampusCart_Auth_v1.postman_collection.json`
Secrets	`.env.example` only

Sample register 201:

```
{"token":"eyJhbGciOiJIUzI1NiJ9...","email":"priya@campuscart.in"}
```

Work from `starter/` (auth TODOs) or `broken/` (debug). Compare `solution/` only after you finish (or when told in class). `solution/` is trainer-only in the student pack.

Stretch

- `@CrossOrigin(origins = "http://localhost:5173")` — **not** Done when. Production origins **Later —**.
- Extra JWT claim — not Done when.
- Authenticated **DELETE** `/products/{id}` — optional.
- Lombok / records — optional, not in Done when.
- **Later —** refresh tokens, OAuth2, admin roles. **Later —** React login. **Later —** protected lab consolidation.

Done when

- [ ] `mvn spring-boot:run` starts (Tomcat + DataSource + Security)
- [ ] Register **201** with token; hash in MySQL (not raw password)
- [ ] Login **200** / bad password **401**
- [ ] GET `/products` **200** without token
- [ ] POST `/products` **401** without token; **201** with Bearer
- [ ] Duplicate email **409**
- [ ] Postman Auth v1 imports; statuses match
- [ ] `JWT_SECRET` and DB password not literals in git
- [ ] No OAuth2 / refresh / Lombok required
- [ ] CORS **not** required to pass
- [ ] `target/` untracked; README with run + curl 401/201 (Standard Project Deliverables)

Interview Q&A

Shared with students after concepts and the mini-project. Use for review and interview prep — questions and model answers are included.

Q1. AuthN vs AuthZ?
A: Authentication identifies the user (login, JWT). Authorization checks permissions (may this user POST `/products`).

Q2. Why BCrypt?
A: Slow salted hash. A stolen table does not reveal raw passwords. Use `encode` on register and `matches` on login.

Q3. What is a JWT?
A: A signed token (header.payload.signature). The server verifies the signature with a secret. The client sends it as `Authorization: Bearer`.

Q4. Why STATELESS?
A: The token carries the login. Tomcat does not store a session for each user. That scales and matches a JSON API.

Q5. Why disable CSRF here?
A: CSRF is mainly a cookie-session browser issue. This design uses a Bearer header the client must set on purpose.

Q6. `WebSecurityConfigurerAdapter`?
A: Gone in Security 6. Configure a `SecurityFilterChain` bean.

Q7. 401 vs 403?
A: 401 = not authenticated. 403 = authenticated but forbidden. This module's core path emphasises 401. Role 403 is later.

Q8. Why keep GET public?
A: A campus menu is readable without an account. Creating a row is a write — that needs a user.

Q9. Where does the secret live?
A: Environment / `.env` locally. Commit `.env.example` with a dummy 32+ character value. Never a real secret in yml on git.

Q10. Why 32 characters?
A: HS256 needs a 256-bit key. jjwt will reject a short secret.

Q11. Can JWT replace HTTPS?
A: No. Tokens in headers are still visible on HTTP. Lab is localhost; production needs TLS (**Later —** deploy lab).

Q12. Does this use OAuth2?
A: No. We issue our own HS256 tokens. Google / Auth0 is a later, larger topic.

Q13. Why `AppUser` not `User`?
A: Avoid clashing with `org.springframework.security.core.userdetails.User`.

Q14. How does this connect to CampusCart?
A: Capstone login/register is this spike. Catalog GET stays public; creates (and later orders) sit behind JWT.

Q15. Is CORS part of Security?
A: CORS is a browser origin rule (module 04 recap). Security is identity and access. Both can exist; CORS is Stretch here.

Next

Push **CampusCart JWT Auth Lite** (kit 19 — collection committed, no secrets). Continue with **Lab: CRUD + Auth Consolidation** — protected lab days. OAuth2, refresh tokens, and React login stay **Later —**.